

# Lecture 22 — Of Asgard and Hel

Jeff Zarnett  
jzarnett@uwaterloo.ca

Department of Electrical and Computer Engineering  
University of Waterloo

August 13, 2026

# Yggdrasil, the World Tree



Everything came into creation in the gap between fire and ice, and the World Tree (Yggdrasil) connects the nine worlds.

Asgard is the home of the Æsir, the Norse gods.

Helheim, or simply Hel, is the underworld where the dead go upon their death.

In Hel or Asgard (it's not clear), there is Valhalla, hall of the honoured dead.

# Carry We, Who Die In Battle...

Those who die in battle and are judged worthy will be carried to Valhalla by the Valkyries.

There they will reside until they are called upon to aid in Odin's fight with the wolf Fenrir in Ragnarök.

humans live in the "middle realm", Midgård, surrounded by the serpent Jormungand, who will fight against Thor in Ragnarök.

Thor will kill the serpent, but the serpent's poison will also finish off Thor.

# Across the Rainbow Bridge, To Valhalla...

We're going to examine some very useful tools for programming called Valgrind and Helgrind (also Cachegrind).

Where do they take their names from? Valgrind is the gateway to Valhalla; a gate that only the worthy can pass.

Helgrind is the gateway to, well, Hel.

I may be interested in this subject...



But all of these are analysis tools for your C and C++ programs.

They are absolute murder on performance, but they are wonderful for finding errors in your program.

To use them, start the tool of your choice and instruct it to invoke your program.

The target program then runs under the “supervision” of the tool.

Remember to enable debugging symbols in your compile.

Valgrind is the base name of the project and by default what it's going to do is run the memcheck tool.

The purpose of memcheck is to look into all memory reads, writes, and to intercept and analyze every call to `malloc/free` and `new/delete`.

# Is Your Program Worthy?



Memcheck can find problems like:

- Accessing uninitialized memory
- Reading off the end of an array
- Memory leaks
- Incorrect freeing of memory
- Incorrect use of C standard functions like `memcpy`
- Using memory after it's been freed.
- Asking for an invalid number of bytes in an allocation

```
==8476== All heap blocks were freed -- no leaks are possible
```

That's the ideal. But is this realistic?

# Send the Plumbers to the Watergate

Take the program's suggestion to use the `-leak-check=full`.

You get a bit more detail about where you made the mistake.

In the example below, lines 49 and 24 in the file `search.c` are the locations of the `malloc` calls that lack a matching call to `free`.

It can't tell you where the call to `free` should go, only where the memory that isn't freed was allocated.

- **Definitely lost**
- **Indirectly lost**
- **Possibly lost**
- **Still reachable**
- **Suppressed**

It's also important to learn what to ignore (or what's out of our hands).

Example: it's thread creation or a library or similar. What do we do?

Consider carefully if we can do something about it!

Let's do some examples.



Credit: Norse Legends Fandom User OlaTansta

The purpose of Helgrind is to detect errors in the use of POSIX pthreads.

In a way, Helgrind is a pretty neat tool for improving performance, even though it doesn't actually directly speed anything up.

When we take a single-threaded program and split it off into a multithreaded program, we may introduce a lot of errors.

Humans are not very good at parallel thinking; we are very much sequential.

But a program that is fast and wrong is probably less useful than one that is slow and correct.

But can we make it faster and still have it be correct?

That's the goal of Helgrind: determine where, if anywhere, there are concurrency problems.

It can't prove that your program is correct (if only) but it can at least catch some of the common problems you might introduce when writing a parallel program.

Helgrind will categorize errors into three types:

- 1 Misuses of the pthreads API
- 2 Lock ordering problems
- 3 Data races

The first category does not require much explanation:

- Unlocking a mutex that is unlocked
- Deallocation of memory with a locked mutex in it
- Thread exit while holding a locked lock

... and many more.

The second category of errors should be familiar to you:

## Thread P

1. wait( a )
2. wait( b )
3. [critical section]
4. signal( a )
5. signal( b )

## Thread Q

1. wait( b )
2. wait( a )
3. [critical section]
4. signal( b )
5. signal( a )

Potential Deadlock!

Risk of deadlock: thread P holds mutex a and thread Q holds mutex b.

Each waits for the mutex that the other one has.

The example is slightly silly, of course, because it's super easy to see.

There will not necessarily be an obvious (alphabetical) order.

Helgrind builds a directed graph of lock acquisitions, so that when a thread acquires a lock, the graph is checked to see if a cycle exists.

It will report as an error the initial order (the first order seen is the one viewed as “correct”) and the “incorrect” order that is the source of the potential problem.

Really, though, all that matters is consistency.

The third category we have discussed already.

Recall the earlier definition of a race condition.

Helgrind looks for when two threads access the same memory location without using locks.

It examines the use of the standard threading primitives - lock, unlock, signal/post, wait...

Anything that implies there might be an ordering between events is taken and added to a directed acyclic graph that represents these dependencies.

If memory is accessed from two different threads and there is no path through this directed acyclic graph that indicates an ordering, then a race is reported.

Although obviously at least one of these accesses must be a write.

You can ask Helgrind to try to tell you about variable names (if it can) with the command line option `-read-var-info=yes`.

```
==10454== Location 0x60104c is 0 bytes inside global var "var"  
==10454== declared at datarace.c:3
```

# Fixing it Isn't My Problem

The authors of Helgrind assume that if it tells you where the problem is, you will figure out what variables are affected and how to properly prevent data races.



You might find this frustrating.

Let's do some more examples.